

IDONTKNOWITSNAME

Anonymous Author(s)

Abstract

LLM agent frameworks increasingly expose model invocations through structured programming abstractions, with framework runtimes lowering program-level constructs into concrete prompts. We observe that these programmatically generated prompts are inherently heterogeneous: their components exhibit different sharing scopes, lifetimes, and evolution patterns across invocations. However, existing serving systems treat prompts as opaque text, losing this structure and missing opportunities for more effective KV-cache reuse and management.

We present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME’s compiler pass performs static analysis and compiler-level optimizations to expose binding lifetimes, expose application-level structure and proactively re-layout prompts to improve cache reuse. The serving runtime exploits the heterogeneity information to optimize KV-cache behavior across requests by prefetching, protection and eviction, according to the predicted future utility of cache entries. We implemented IDONTKNOWITSNAME on SGLang and evaluated it against multiple established serving systems. Our results shows that IDONTKNOWITSNAME.....

1 Introduction

Agent frameworks increasingly expose LLM calls as programming abstractions rather than manual prompt construction. Systems such as ByLLM [3], NVIDIA Object-Oriented Agents (NOOA) [5], and DSPy [7] allow developers to describe model behavior using functions, objects, signatures, and modules, which framework runtimes lower into concrete prompts. This approach also appears in systems such as SIGIL [4], which compiles natural-language skill documents into programs containing typed LLM calls. LLM requests from these frameworks are generated programmatically: they are the serializations of program state rather than free text.

We observed that these programmatically generated LLM requests are inherently heterogeneous. The heterogeneity arises from three dimensions: (1) Sharing scope: one value can be shared across multiple invocations, or only within a single invocation. (2) Lifetime: Different bindings have different lifetimes. Some bindings are single-shot volatile inputs, while others may be reuse across the whole session. (3) Evolution pattern: context for LLM calls can evolve differently overtime, such as remaining fixed, being replaced, or accumulating across invocations. Figure 1 illustrates this heterogeneity in a HoVer-style [6] fact check agent. The claim persists throughout a verification session, the evidence list accumulates over time, and each search result is consumed

once and replaced. Although these values appear in the same model invocation, they offer substantially different reuse opportunities.

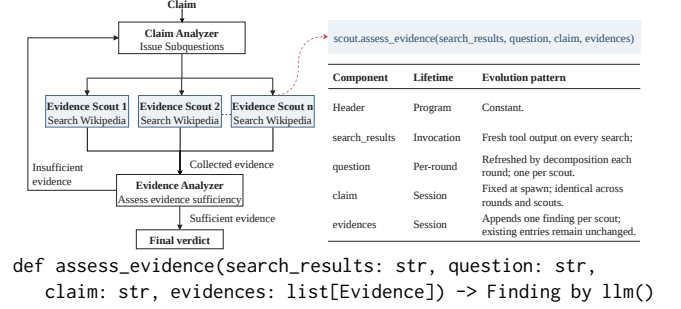


Figure 1. Heterogeneity within a HoVer-style [6] Fact Check Agent’s generated prompt for the summary-making function. `assess_evidence(search_results, question, claim, evidences)` evaluates the sub-question against the accumulated evidence and search results, then answers it.

This distinction matters because main-stream KV-cache serving backbones are prefix-based. In the example, placing the accumulating evidences binding after the short-lived `search_results` binding prevents the model from reusing the evidence prefix whenever the search result changes. The model must therefore recompute KV states for tokens whose contents remain unchanged. Moreover, without knowing which components persist or evolve, the serving system cannot accurately determine which cached tokens should be retained, evicted, or prefetched.

However, existing agent serving systems primarily optimize at the level of requests or agent invocation sequences. Pythia [9] profiles agent workflows to guide cache management, request scheduling, and autoscaling. CacheScout [10] learns execution transitions online to guide eviction and prefetching, while PBKV [11] predicts future invocations and estimates the reuse potential of cached requests. These systems exploit predictability between invocations, but treat the contents of each prompt as an opaque token sequence. Consequently, they cannot identify the heterogeneity within a single prompt, leading to a suboptimal serving performance for such program-integrated agents.

To close the gap, we present IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management. IDONTKNOWITSNAME achieves this by exposing additional prefix reuse and keeping the resulting KV states resident when they are likely to be needed. The identification and exploitation of prompt heterogeneity introduces the following two challenges:

First, heterogeneity is not always explicit in prompt. The sharing scope and lifetime of bindings are closely bonded with the program-level structure and actual runtime assignment. Determining which layout best serves prefix caching is therefore not trivial. IDONTKNOWITSNAME combines static analysis with runtime information to determine prompt layouts. The compiler derives a partial order over prompt components based on their sharing scopes and lifetimes. Then, IDONTKNOWITSNAME uses the actual token lengths of bindings observed from log to order components left unconstrained by this partial order, producing a concrete layout that improves prefix cache efficiency.

Second, exploiting future reuse requires coordinating cache retention and prefetching. Prompt components differ in when they will be reused and how much recomputation their cached prefixes can avoid. Retaining these prefixes consumes capacity needed by other requests, while prefetching too early occupies device memory and prefetching too late fails to hide transfer latency. Inappropriate KV-cache eviction would lead to unnecessary recomputation. IDONTKNOWITSNAME addresses this challenge through two coordinated planners: a protection planner prioritizes cached prefixes according to their expected reuse and benefit, while a prefetch planner schedules transfers based on predicted invocation deadlines and available device memory.

We implemented IDONTKNOWITSNAME on SGLang [12].... We evaluated IDONTKNOWITSNAME with three representative agent workloads: HoVer-style [6] fact check agent, HumanEval [2] code agent, and a BFCL [8] tool-use agent.... Our evaluation shows that IDONTKNOWITSNAME...

In summary, we make the following contributions.

- We identify the inherent heterogeneity in programmatically generated prompts and its implications for serving.
- We design and build IDONTKNOWITSNAME, a compiler-serving co-design that recovers and exploits prompt heterogeneity for efficient KV-cache management....
- We conduct comprehensive evaluations of IDONTKNOWITSNAME against multiple established serving systems, demonstrating that IDONTKNOWITSNAME achieves up to...

2 Background

2.1 Serving systems for LLM agents

2.2 Agent frameworks with LLM calls as programming abstractions

3 Motivation

4 Design

4.1 Overview

In this section, We present the design of IDONTKNOWITSNAME, an efficient serving engine for program-integrated

LLM agents. IDONTKNOWITSNAME adopts a two-tier compiler-server co-design. On the compiler side, IDONTKNOWITSNAME performs static analysis to gather heterogeneity metadata, and on the server side, IDONTKNOWITSNAME leverages this metadata to optimize KV-cache scheduling. We first introduce an interface *Agent Descriptor* to bridge the compiler and server, together with the methodology of their construction. Second, we present a heterogeneity-aware KV-cache scheduling strategy that leverages the agent descriptor to enhance serving performance.

4.2 Agent Descriptor for Heterogeneity Metadata

Here we define *Agent Descriptor* to bridge static analysis and server-side scheduling. An agent descriptor contains two parts: (i) **Call-site Template**: It provides a heuristic of the components' lifetime, sharing scope and evolution pattern. (ii) **Workflow Automaton**: It contains the program-level structure of an agent, including workflow dependencies, execution time estimation and transition probabilities. IDONTKNOWITSNAME's two-tier design let the compiler to provide a skeleton, while the server refines the weight with runtime observations. Two sides are therefore connected by this agent descriptor, and jointly serve a more efficient KV-cache scheduling strategy.

4.2.1 Call-site Template. As discussed in §3, the components of a programmatically generated prompt fall into two categories: (i) **Invariant Specifications**: the components that are fixed once the program is compiled, such as the system prompt, response schema, and the tool declarations. They are rendered from the compiler's own intermediate representation and stays constant. (ii) **Dynamic Bindings**: because LLM calls are declared as ordinary functions, the arguments of a call site are supplied in runtime. These runtime values are the dynamic bindings.

We therefore define the call-site template as

$$\mathcal{T}_c = \langle \mathcal{I}_c, \mathcal{B}_c, \mathcal{H}_c \rangle,$$

where $\mathcal{I}_c$ is the invariant specifications, $\mathcal{B}_c$ is the dynamic bindings, and $\mathcal{H}_c$ is the heterogeneity annotation over those bindings. All three are recovered by static analysis.

Heterogeneity annotation. For each dynamic binding, $\mathcal{H}_c$ records both how its value evolves from one call to the next and how long that value stays valid:

$$\mathcal{H}_c : \mathcal{B}_c \longrightarrow \mathcal{E} \times \mathcal{S}.$$

The evolution component $\mathcal{E}$ classifies a binding by its value provenance. We distinguish $\mathcal{E}$ into five classes, listed from the most to the least reusable:

1. **CONST**: the value is fixed by the program text and is identical at every call.
2. **COPY**: the value is the same as the value another call site carries, so the bytes are already resident once that call site has run.

3. **RESP**: the value comes from the reply produced by another call site.
4. **EXTEND**: the binding is an append-only context, and the previous value remains a prefix of the new one.
5. **VOLATILE**: the value is freshly produced at each call, or its origin lies outside what the analysis can see.

We write ρ for the rank of a class in the order above, which expresses how much of a binding's bytes an earlier call can already supply.

The lifetime component $\mathcal{S}$ names the program scope whose completion invalidates a binding's value ($\mathcal{S} = \emptyset$ meaning that the value survives for the whole session once produced). For example, `claim` in `scout.assess_evidence` in figure 1 has $\mathcal{E} = \text{CONST}$ and $\mathcal{S} = \emptyset$. Lifetime is independent of evolution, and the server needs both. The evolution class governs which bytes a later prompt can reuse, while the lifetime governs when cached bytes can be given up.

4.2.2 Workflow Automaton. An agent typically consists of a sequence of dependent LLM calls whose workflow structure is implicit in the program. Concretely, we abstract an agent as a time-annotated probabilistic finite-state machine (TPFM). We formulate the TPFM as:

$$\mathcal{A} = (Q, P, q_0, F, \mathcal{T}),$$

where:

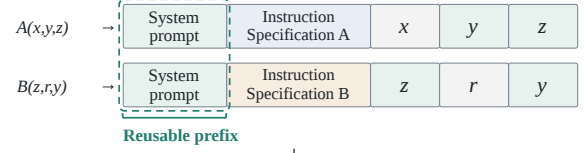
1. Q is the set of states. Each state $q \in Q$ is labelled by an LLM callsite;
2. P denotes the transition probabilities between states.
3. $q_0 \in Q$ is the entrance callsite.
4. F specifies the termination probability of the agent at each state.
5. $\mathcal{T} = \{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\}$ denotes the timing annotations associated with states and transitions, where D_q^{exec} is the execution-time distribution of the LLM callsite represented by state q , and $D_{q,q'}^{\text{gap}}$ is the inter-call gap distribution between the completion of callsite q and the invocation of its successor q' .

Static analysis fixes the shape of this automaton, while its weights comes from runtime logs. The states, the entry, the transitions among them, and final states are recovered from code. $\mathcal{T} = \{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\}$ and P are often unknown at compile time and model dependent. The server observes these online and accumulates them across sessions, updating the statistical model of the agent consistently. Together the two halves give the server a forward-looking view of a session.

4.3 Heterogeneity-aware KV-cache Arrangement

4.3.1 Adaptive Prompt re-layout. Because prefix cache reuse stops at the first byte that differs. The message a framework emits for a call site follows the declaration order of the function, therefore may disable the potential of prefix reuse. IDONTKNOWITSNAME's adaptive re-layout permutes

(a) Original layout



(b) Reordered layout

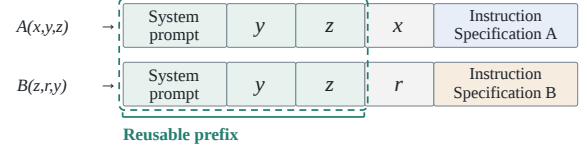


Figure 2. Example of adaptive re-layout, which increases prefix reuse by moving the more reusable components to the front, exposing shared prefix to the KV-cache.

the components of each call message so that the bytes most likely to be resident already come first, which enables an improved prefix reuse. The layout permutation's basic is the compiler derives a partial order over components, as digested in template $\mathcal{T}_c$ 4.2.1, server then freezes the re-layout verdict with observed token length from runtime.

Transportable Components. First, the dynamic bindings $\mathcal{B}_c$ can be permuted freely. The compiler lowers each argument into a self-contained component labelled by its parameter name, and the framework places these components in declaration order only because that is the order of the argument list; no construct of the program fixes their relative position. Re-layout may therefore choose any order of the bindings.

Second, part of the invariant specification $\mathcal{I}_c$ is movable as well. Invariant prompt $\mathcal{I}_c$ in our declaration is a simplified abstraction. More precisely, it consists system prompt, reponse schema, tool schema and instruction specification (Task description and parameter explanation generated by compiler). Among them, system prompt stays universally constant, and two schemas are digested by model separately. Instruction header is movable.

IDONTKNOWITSNAME therefore operates on the order of bindings and the placement of instruction specification. The following paragraphs describe how IDONTKNOWITSNAME chooses a layout within this space.

Layout decision algorithm. The layout of a call site is a program-wide decision made from the compiler-derived template $\mathcal{T}_c$. The template alone yields only a partial order, so the server observes the token length of each binding's value at runtime to break the ties within each band. Concretely, the bindings of a call site are sorted in ascending lexicographic

order of the tuple

$$K(b) = (\text{shared}, \text{scoped}, \rho, \text{depth}, -\text{length}),$$

whose components are defined as follows:

- *shared*: whether the value of b is carried by more than one call site; 0 if shared and 1 otherwise, so shared bindings lead.
- *scoped*: the lifetime of b ; 0 if it is session-wide ($\mathcal{S} = \emptyset$) and 1 otherwise, so bindings that survive scope boundaries precede those reset by them.
- ρ : the stability rank defined in §4.2.1, from 1 for CONST to 5 for VOLATILE, so the more reusable class leads.
- *depth*: the number of transitions in $\mathcal{A}$ (§4.2.2) from q_0 to the call site at which the value of b first appears, so values that enter a session earlier lead and the prompt of an upstream call site becomes a prefix of those downstream.
- *length*: the token length of the value of b , observed by the server; negated so that longer bindings stay in front and a change in a shorter binding invalidates fewer cached tokens.

The first four components are fixed by the program and constitute the compiler's partial order; only *length* is observed in at runtime. For a binding shared by several call-sites, the value of *scoped* and ρ will be evaluated among all call-sites that carry it to make sure consistency of ordering across call-site. Specifically, the value is marked as *scoped* if any of the call-sites bounds its lifetime, and its stability ranking is **EXTEND** if any of them appends to it and **COPY** otherwise.

specification placement. The *instruction specification* is lowered from code by compiler, which is unique for each call-site. Consequently, emitting it first would prevent prefix sharing among different call-sites, however many values they have in common. After sorting, IDONTKNOWITSNAME moves the specification behind the bindings whenever the leading binding is shared, so that sibling call sites can share large blocks of common bindings. A per-program switch is additionally designed to keep the specification in front when moving it is suspected of changing the model's behavior.

As shown in Algorithm 1, IDONTKNOWITSNAME first computes the sorting tuple of every binding, then performs a per call-site sort to obtain its final layout.

4.3.2 Proactive KV-cache Scheduling. While adaptive prompt re-layout improves prefix reuse only by reshaping the request, IDONTKNOWITSNAME's KV-cache scheduling proactively proactively arrange the placement of KV cache entries. Using the agent descriptor, the server decides for every served KV-cache sequence what to keep, what to give up, and what to load back ahead of time.

Identify Reusable KV-cache. Since re-layout already places the stable and shared bindings at the front, a served prompt naturally divides into two segments: a reusable head

Algorithm 1 Layout decision Program

Require: Per call-site templates $\{\mathcal{T}_c\}$;
observed lengths ℓ ;
specification-pinning switch *pin*;
Ensure: Layout L_c for each call site c

```

1: for all  $b \in \bigcup_c \mathcal{B}_c$  do
2:    $K(b) \leftarrow (\text{shared}, \text{scoped}, \rho, \text{depth}, -\ell(b))$ 
    $\triangleright$  Assign sorting tuple for each binding
3: end for
4: for all call sites  $c$  do
5:    $\pi_c \leftarrow \text{SORT}(\mathcal{B}_c, K)$   $\triangleright$  ascending lexicographic
6:    $s_c \leftarrow \text{instruction specification from } \mathcal{I}_c \in \mathcal{T}_c$ 
7:   if  $\neg \text{pin} \wedge (\exists b \in \mathcal{B}_c : K(b)[\text{shared}] = 0)$  then
8:      $L_c \leftarrow \pi_c \parallel \langle s_c \rangle$   $\triangleright$  Expose the shared prefix
9:   else
10:     $L_c \leftarrow \langle s_c \rangle \parallel \pi_c$   $\triangleright$  Keep the specification first
11:   end if
12: end for
13: return  $\{L_c\}_c$ 

```

and a volatile tail. In a full prompt, what behind the reusable head are all classified into volatile tail.

Reusable head consists two parts: a *static prefix* and a *session-shared extension*:

- **Static Prefix** is what the call site emits without run-time inputs, and its values outlives single session. It includes the invariant specifications $\mathcal{I}_c$ together with the CONST bindings that re-layout has placed right behind them.
- **Session-shared Extension** is the run of bindings behind the static prefix that a later call of the same session will emit again at the same position.

The two parts are derived from agent program itself. $\mathcal{H}_c$ names the provenance of each binding, and the automaton $\mathcal{A}$ names the transitions between call sites. Additionally with the decided layout of each call-site, we determine the reusable head by jointly analyze them.

Predict Future Reuse. Because the workflow automaton $\mathcal{A}$ already exposes all potential next calls, our prediction focus on the following three aspects: (i) The probability of the arrival. (ii) When will the call arrive. (iii) Which prefix of its prompt is already known.

Probability of arrival. We estimate the probability of the next call by a loop-tagged table. As the compiler side emits the workflow graph, IDONTKNOWITSNAME adopts Bourdoncle-style [1] weak topological ordering to discover all loops within the agent, so that each call-site q is assigned the chain of loops enclosing it. The server counts the iterations of each loop per session, so every call carries a tuple ζ of the current iteration numbers of the loops enclosing its call-site, outermost first. We refine the transition and termination

probabilities of $\mathcal{A}$ by this tuple and define the table as

$$\mathcal{P}(q' | q, \zeta) \quad \mathcal{F}(q, \zeta)$$

where ζ is the loop indices; $\mathcal{P}$ stands for the probability that a call-site q in context ζ is followed by a call-site q' ; $\mathcal{F}$ stands for the probability that the session ends at call-site q under context ζ . With an empty ζ they reduce to P and F of §4.2.2.

The table is a frequency count that the server updates across sessions. When a call arrives, its call-site is counted as the outcome of the previous call-site under the previous call's ζ , and termination is counted when the session closes. Each observation is also counted under every suffix of ζ down to the empty tuple, so that for the same q the table holds one row per level of context, from the fully specified iteration to the pooled count over all iterations. After a call at q lands, we compute the probability of each outcome o by Witten–Bell interpolation from the empty context ζ_0 to the full context $\zeta_m = \zeta$, where o is a successor of q or a session termination:

$$\hat{P}_k(o) = \lambda_k \frac{c_k(o)}{n_k} + (1 - \lambda_k) \hat{P}_{k-1}(o),$$

$$\hat{P}_0(o) = \frac{1}{|\text{succ}(q)|}, \quad \lambda_k = \frac{n_k}{n_k + d_k},$$

where $\text{succ}(q)$ is the set of static successors of q (including termination, which is treated as at an exit call-site). ζ_k is the suffix of ζ that keeps only the innermost k indices, and the three counts are read from the row (q, ζ_k) of the table: $c_k(o)$ is how many times outcome o followed a call at q in context ζ_k , $n_k = \sum_o c_k(o)$ is the number of observations in that row, and d_k is the number of distinct outcomes with $c_k(o) > 0$.

Then,

$$\mathcal{P}(q' | q, \zeta) = \hat{P}_m(q') \quad \mathcal{F}(q, \zeta) = \hat{P}_m(\text{end})$$

A level with no observation is skipped. The weight λ_k grows with the support of a level, so a well-observed iteration dominates the prediction while a sparse one defers to the pooled behavior of q .

Estimate call arrival time. The time of arrival is conservatively approximated based on record statistics. The workflow automaton records $D_{q,q'}^{\text{gap}}$ per transition: each time a call at q' arrives, the server appends the interval between the reply of the preceding call at q and this arrival. The interval is the agent's own time spent in its code between two calls.

We estimate the gap before the next call as:

$$\hat{g} = t_{50} - \frac{1}{2}(t_{90} - t_{50})$$

where t_{50} and t_{90} are the 50th and 90th percentiles of the gap samples on the transition (q, q') ; the predicted call is then expected $\hat{g}$ after the reply of the current call. We introduce the second term to make sure a transition with more sparse distribution of gap receives a tighter estimation.

Construct prompt ahead of time. As indicated in 4.3.2, the prompt of the known successor q' is reconstructible, or at least partial-reconstructible. IDONTKNOWITSNAME builds

it prompt in the layout $L_{q'}$, using the values that the session already carries. The construction processes bindings in the order of $L_{q'}$ while consulting their evolution class, deducing value accordingly. The construction stops at the first binding with no known origin. Then the matched prefix is sent to kv-planner for speculative preparation.

Lifecycle aware KV-cache scheduling. As we discussed in 3, different components have different lifetime, sharing scope and evolution patterns. Therefore, relying on recency based LRU would lead to suboptimal performance. IDONTKNOWITSNAME adopts a banded priority and steers KV-cache upon it.

Definition of Priority Bands. We categorize KV-cache entries into four bands, from first evicted to last:

1. **RETIRED:** the volatile tail of prompt belonging to completed sessions.
2. **TRANSIENT:** the volatile tail of a served prompt of a live session. The session that produced them will not present them again.
3. **UNCLASSIFIED:** entries that analysis cannot categorize, such as the prompts of a program that was never registered. They keep the engine's default treatment.
4. **PLANNED:** prefixes that a predicted call will reuse. Inside this band entries are ordered by the expected value of keeping them resident, defined next.

Eviction empties the lowest band first and falls back to recency only within a band. Generally, KV-cache that will likely be reused left memory later.

Band Based KV-cache scheduling. As we mentioned above, eviction empties the lowest band first and falls back to recency only within a band, so KV-cache that will likely be reused left memory later. On the host tier, however, it keeps its recency order except that retired segments go first, so a head the device gave up under pressure remains reloadable while it may still be needed. Additionally, KV-cache entries marked as *TRANSIENT* won't be admitted into host as they rival memory space with reusable KV-cache entries.

Bands are assigned at three moments of a session's lifecycle. (1) **When a served prompt enters the cache**, IDONTKNOWITSNAME splits it at its reusable head (Section 4.3.2). The volatile tail and the reply generated after it become *TRANSIENT*; the head stays *UNCLASSIFIED* until the next prediction refresh finds a successor call that starts with it, which promotes it to *PLANNED*. (2) **When a later call arrives**, the transition it takes may rewrite bindings that earlier heads carried. Each affected head is cut back to the prefix that survives the rewrite, and the bytes past it are demoted to *TRANSIENT*. (3) **When the session ends**, the volatile tails of every prompt it served drop to *RETIRED*. Their heads stay *UNCLASSIFIED* and age out by recency, since other sessions of the same program may still share them.

These bands are mirrored directly into the radix tree's eviction order. The band is the primary key of the eviction heap and recency breaks ties.

Prediction-based KV-cache protection Based on prediction, kv-planner of IDONTKNOWITSNAME conducts protection. The duty of it is to ensure required prefix resides on the device, or at least on host. Each prediction is summarized into a tuple of reconstructed prefix, arrival probability and estimated deadline:

$$\text{Prediction} = (\pi_{q'}, \mathcal{P}(q' | q, \zeta), t_{\text{reply}} + \hat{g})$$

where $\pi_{q'}$ is the prefix reconstructed for q' and t_{reply} is the time the current call replied. Predictions are refreshed at every call, and are dropped when the predicted calls arrive or pass their deadline. Protection acts on this tuple in two ways. *Retention* keeps $\pi_{q'}$ from leaving the device, and *prefetch* restores the part of $\pi_{q'}$ that has already been demoted to the host.

Retention. The planner ranks the predicted reusable KV-cache entries by the value of keeping $\pi_{q'}$ resident at every refresh:

$$\mathcal{D}(q') = \frac{\mathcal{P}(q' | q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}}{\max(t_{\text{reply}} + \hat{g} - t_{\text{now}}, \tau_{\text{min}})}$$

where $|\pi_{q'}|_{\text{cached}}$ is the number of tokens of $\pi_{q'}$ cached (both on host and device) and τ_{min} is a floor that keeps a call due now from dominating. IDONTKNOWITSNAME's kv-planner sort all predictions by $\mathcal{D}$ in descending order, and admits them into *PLANNED* until the band covers a fixed fraction of the device tier. Within the band, their $\mathcal{D}$ add up on every radix cache node, making shared head outranks a private one.

Prefetch. When part of $\pi_{q'}$ has been demoted to the host, the planner prefetches it back to the device ahead of the estimated arrive time. A prefetch release deadline is therefore defined as:

$$t_{\text{release}} = t_{\text{reply}} + \hat{g} - \frac{|\pi_{q'}|_{\text{host}}}{B_{\text{h2d}}} - \frac{|\pi_{q'}|_{\text{uncached}}}{B_{\text{prefill}}}$$

where B_{h2d} and B_{prefill} are the measured host-to-device copy rate and prefill rate. Among released predictions the earliest deadline goes first, and break ties by expected saving:

$$\text{Saving} = \mathcal{P}(q' | q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}$$

A prefetch takes only free device space after reclaiming *RETIRED* and *TRANSIENT* entries, so it never interferes with real requests; when no space is free, prefetching pauses briefly and the prediction keeps its retention. A prefetch is released when the engine is idle or its deadline t_{release} approaches.

5 Related Work

6 Conclusion

The inherent structure and heterogeneity of programmatically generated prompts create opportunities for optimization. IDONTKNOWITSNAME identifies and exploits these opportunities rather than treating each prompt as a monolithic unit, thereby unlocking proactive optimizations that improve performance and efficiency. Our evaluation demonstrates that IDONTKNOWITSNAME's methodology successfully creates a more efficient LLM serving system.

References

- [1] François Bourdoncle. 1993. Efficient Chaotic Iteration Strategies with Widenings. In *Formal Methods in Programming and Their Applications (Lecture Notes in Computer Science, Vol. 735)*. Springer, 128–141. doi:10.1007/BFb0039704
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
- [3] Jayanaka L. Dantanarayana, Yiping Kang, Kugesan Sivasothyathan, Christopher Clarke, Baichuan Li, Savini Kashmira, Krisztian Flautner, Lingjia Tang, and Jason Mars. 2025. MTP: A Meaning-Typed Language Abstraction for AI-Integrated Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1176–1204. doi:10.1145/3763092
- [4] Jayanaka L. Dantanarayana, Savini Kashmira, Lingjia Tang, and Jason Mars. 2026. SIGIL: Compiling Agent Skills into Typed Harnesses. *arXiv:2607.27309 [cs.SE]* <https://arxiv.org/abs/2607.27309>
- [5] Paul Furgale, Severin Klingler, James Nolan, Matt Staats, Gaia Di Lorenzo, Elisa Martinez Abad, Christian Schüller, Razvan Dinu, Alessio Devoto, Pascal Berard, Gal Kaplun, Elad Sarafian, Riccardo Roveri, Leon Derczynski, and Ricardo Silveira Cabral. 2026. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents. *arXiv:2607.20709 [cs.AI]* <https://arxiv.org/abs/2607.20709>
- [6] Yichen Jiang, Shikha Bordia, Zheng Zhong, Charles Dognin, Maneesh Singh, and Mohit Bansal. 2020. HoVer: A Dataset for Many-Hop Fact Extraction And Claim Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 3441–3460.
- [7] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*.
- [8] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International*

- Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267). PMLR, 48371–48392.
- [9] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. 2026. Pythia: Exploiting Workflow Predictability for Efficient Agent-Native LLM Serving. arXiv:2604.25899 [cs.MA] <https://arxiv.org/abs/2604.25899>
- [10] Rui Zhang, Chaeun Kim, Shaoting Feng, Kuntai Du, Yuhua Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. 2026. Learning Agent Execution for KV-Cache Management in Agentic Serving. arXiv:2608.14624 [cs.AI]
- [11] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-based KV-Cache Management. arXiv:2605.06472 [cs.LG] <https://arxiv.org/abs/2605.06472>
- [12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.